```python
"""
author: Sanved
description: notes on python programming: chapter 1
"""

# programming in Python

# introduction

# the print statement
# syntax: print(value1, value2, ..., sep=" ", end="\n")
# value1, value2, ...: the values to be printed
# sep=" ": the separator between the values
# end="\n": the character to be printed at the end of the line

print("Hello world!", "I am Sanved.", sep=" ", end="\n")
# Hello world! I am Sanved.
print("Hello", "world!", sep=", ", end="!!!\n")
# Hello, world!!!!


# variables
# variables are used to store data
x = 5       # integer   # in Python, variables are dynamically typed
y = "10"    # string    # the type of the variable is determined by the value assigned to it
z = x + int(y)          # as opposed to C where it would be int x = 5; char y = '10'; int z = x + y;
print(z)
# 15

# type function
print(type("Hello"))
# <class 'str'>

# id function
# the id function returns the memory address of an object
# syntax: id(object)
x = 5
print(id(x))
# 11754024
y = x
print(id(y))
# 11754024

# references
# in Python, variables are references to objects in memory
# when we assign a variable to another variable, we are copying the reference, not the object itself
# this means that changing the object through one variable will affect the other variable as well
a = [1, 2, 3]
b = a
b[0] = 5
print(a)
# [5, 2, 3]

# memory management
# Python uses reference counting to manage memory
# when an object is created, its reference count is set to 1
# when a reference to the object is created, the reference count is incremented
```

```python
# when the reference goes out of scope, the reference count is decremented
# when the reference count reaches 0, the object is deleted from memory
# Python also uses a garbage collector to handle circular references
a = 10
b = a
c = b
del a
print(c)
# 10

# rules for variable names
# start with a letter or an underscore (not two underscores), not a digit
# followed by letters, digits, or underscores, no spaces or special characters
# case-sensitive
# cannot be a keyword (reserved word)
# examples: x, _x, x1, x_1, X, _X, X1, X_1 are valid
# examples: 1x, x-1, x 1, x@1, x#1, x~1, if, *x are invalid

# input function
# input(prompt)
x = input("Enter a number: ")
print(x)
# Enter a number: 5
# 5

# input in a line
x, y = input("Enter two numbers: ").split()
print(x, y)
# Enter two numbers: 5 10
# 5 10

# input in a line with type casting
x, y = map(int, input("Enter two numbers: ").split())
print(x, y)
# Enter two numbers: 5 10
# 5 10

# type casting
# syntax: int(value), float(value), str(value)
x = "5"
y = int(x)
```